Hospital Management System (HMS) — Professional Documentation

1. Project Overview

The **Hospital Management System (HMS)** is a comprehensive, enterprise-grade healthcare management platform designed to streamline clinical and administrative operations. It provides a unified interface for healthcare providers to manage patient care, scheduling, logistics, and financial workflows.

Purpose

To modernize hospital operations by replacing fragmented manual processes with a centralized, secure, and highly efficient digital ecosystem.

Problem it Solves

- **Data Silos**: Consolidates patient records, billing, and pharmacy data.
- **Operational Inefficiency**: Automates appointment scheduling and inventory tracking.

- **Security Risks**: Implements strict Role-Based Access Control (RBAC) to protect sensitive health information (PHI).

- **Inaccurate Reporting**: Provides real-time dashboards for hospital performance metrics.

Main Features

- **Patient Directory**: Complete Electronic Medical Records (EMR) management.

- **Dynamic Scheduling**: Intelligent appointment booking with status tracking.

- **Pharmacy & Lab Management**: Inventory tracking and diagnostic test monitoring.

- **Revenue Cycle Management**: Professional invoicing and automated billing.

- **Facility Logistics**: Bed management and staff scheduling.

- **Emergency Blood Bank**: Real-time inventory and donation tracking.

Target Users

- **Administrators**: Hospital management and operational oversight.
- **Medical Staff**: Doctors, Nurses, and Lab Technicians.
- **Administrative Staff**: Receptionists and Billing Officers.
- **Patients**: Appointment self-booking and record access.

---

2. Tech Stack

Frontend Technologies

- **React 19**: Modern UI framework with Concurrent Rendering.
- **Vite**: Ultra-fast build tool and development server.
- **Bootstrap 5**: Responsive design system and layout components.

- **Boneyard-js**: Custom metadata-driven UI library for dynamic component generation.

- **React Router 7**: Sophisticated SPA routing and navigation.

Backend Technologies

- **FastAPI**: High-performance Python web framework based on ASGI.

- **SQLAlchemy 2.0**: Advanced ORM for database abstraction.

- **Pydantic**: Robust data validation and settings management.

- **Uvicorn**: Lightning-fast ASGI server implementation.

Database

- **SQLite**: Local development and light production.

- **PostgreSQL**: Recommended for high-concurrency production environments (fully supported).

APIs & Services

- **RESTful API**: Stateless communication between frontend and backend.

- **WebSockets**: Real-time updates for dashboard statistics and alerts.

- **n8n / Webhooks**: External integrations for notification services.

Deployment Tools

- **Docker & Docker Compose**: Containerization for consistent environment parity.

- **Alembic**: Database migration management.

---

3. Project Architecture

High-Level Architecture

The HMS follows a **Decoupled Client-Server Architecture**. The frontend is a static single-page application (SPA) that communicates with a centralized

RESTful API backend.

Frontend Flow

1. **Navigation**: Handled by React Router with code-splitting (lazy loading).
2. **State Management**: Centralized via `AppContext` for user sessions and global UI state.
3. **UI Rendering**: Uses a "Bones" metadata system where JSON definitions drive the rendering of complex tables and grids.

Backend Flow

1. **Entry Point**: `main.py` initializes the FastAPI application.
2. **Middleware**: Handles CORS, Security Headers (CSP, HSTS), GZip compression, and Rate Limiting.
3. **Routing**: Request is routed to specific modules (e.g., `/api/patients`).
4. **Authentication**: JWT verification via dependency injection.
5. **Logic**: CRUD operations performed using SQLAlchemy sessions.

Data Flow

`User Action` → `API Service (Frontend)` → `FastAPI Router` → `Service/Logic Layer` → `SQLAlchemy ORM` → `Database` → `JSON Response` → `UI Update`

---

4. Folder Structure Explanation

Root Directory

- `/backend`: Complete Python backend application.
- `/src`: React frontend source code.
- `/public`: Static assets (logos, icons).
- `/landing`: Marketing landing page source.
- `/dist`: Optimized production build output.
- `package.json`: Frontend dependencies and scripts.
- `docker-compose.yml`: Multi-container orchestration.

Backend Structure (`/backend/app`)

- `/core`: Global configurations, database setup, and security utilities.
- `/routers`: API route definitions grouped by resource.
- `/modules`: Feature-specific logic (Auth, Patients) containing their own models and schemas.

- `models.py`: Centralized database schema definitions.

- `main.py`: Application initialization and middleware setup.

Frontend Structure (`/src`)

- `/components`: Reusable UI elements (Layout, Auth, Common).

- `/pages`: Page-level components corresponding to routes.

- `/lib`: Core utilities like the `api.js` client.

- `/bones`: JSON metadata files defining the structure of various system tables.

- `/context`: Context API providers for global state.

---

5. Frontend Documentation

Pages & Components

- **Dashboard**: High-level statistics and real-time activity feed.
- **Patients**: Master directory with advanced search and filtering.
- **Appointments**: Calendar-style management and booking.
- **Staff/User Management**: Admin-only controls for personnel.

Routing

Implemented in `App.jsx` using `react-router-dom`. Routes are wrapped in a `ProtectedRoute` component that enforces authentication and role-based permissions.

State Management

The `AppContext` handles:

- **User Session**: Login state and profile info.
- **Auth Tokens**: Management of JWT in `sessionStorage`.
- **System Theme**: Dark/Light mode preferences.

API Integration

Managed through `src/lib/api.js`. It uses the `fetch` API with automated:

- **Bearer Token Attachment**: Injects JWT into protected requests.
- **Error Normalization**: Maps backend Pydantic errors to user-friendly messages.
- **Response Handling**: Centralized JSON parsing and 401/403 redirection.

Styling Approach

- **Vanilla CSS**: Used for custom branding and micro-animations.
- **Bootstrap Utilities**: Used for layout grid and standard UI components.
- **Glassmorphism**: Applied to cards and sidebars for a premium aesthetic.

---

6. Backend Documentation

Server Setup

Uses **FastAPI** with `lifespan` events for database initialization and seeding.

Middleware

- **CORSMiddleware**: Restricts API access to authorized domains.
- **Secure (CSP/HSTS)**: Hardens the app against XSS and MITM attacks.
- **SlowAPI**: Implements rate limiting to prevent brute-force attacks.
- **GZip**: Compresses large JSON payloads for faster performance.

Controllers & Logic

Unlike traditional MVC, logic is shared between **Routers** (handling HTTP specifics) and **CRUD utilities** (handling database interactions).

Error Handling

A global exception handler in `main.py` catches all unhandled Python errors and returns a sanitized JSON response, preventing stack trace leaks in production.

---

7. Database Documentation

The system uses a relational schema designed for integrity and query performance.

Key Tables

Table	Description	Important Fields
users	System identities	`username`, `email`, `role`, `password_hash`
patients	Clinical records	`patient_code`, `name`, `age`, `gender`, `status`
appointments	Scheduling	`date`, `type`, `status`, `doctor_name`
invoices	Billing	`invoice_code`, `amount`, `status`, `line_items` (JSON)
medicines	Pharmacy	`name`, `batch`, `stock`, `expiry_date`
staff	HR management	`staff_code`, `role`, `department`, `shift`

Relationships

- **Owner-based Isolation**: Most records include an `owner_user_id` to support multi-tenancy or audit trails.

- **Implicit Links**: Resources are linked via unique codes (e.g., `patient_code`) for flexibility across storage engines.

---

8. API Documentation

Authentication

- `POST /api/auth/login`: Authenticate and receive JWT.

- `GET /api/auth/me`: Verify session and get user profile.

- `POST /api/auth/logout`: Server-side token revocation.

Clinical Modules (CRUD)

Standard patterns apply: `GET` (List), `POST` (Create), `PUT` (Update), `DELETE` (Remove).

- `/api/patients`
- `/api/appointments`
- `/api/records` (EMR)
- `/api/tests` (Lab)

Administrative Modules

- `/api/invoices`: Includes `POST /send-paid-email` and `/download-pdf`.
- `/api/inventory`: Medical supplies management.
- `/api/beds`: Ward and bed availability tracking.

---

9. Authentication & Authorization

Login Flow

1. User submits credentials to `/auth/login`.
2. Backend validates password hash (Argon2/Bcrypt).
3. Backend returns a JWT signed with `HMS_SECRET_KEY`.
4. Frontend stores JWT in `sessionStorage` and updates `AppContext`.

Protected Routes

The `ProtectedRoute` component checks for a valid user in context. If a route has `allowedRoles` (e.g., `['Admin', 'Doctor']`), it verifies the user's role before rendering the component.

---

10. Role-Based Access Control (RBAC)

The HMS implements a granular RBAC system to ensure data privacy and operational security. Permissions are enforced both at the UI level (conditional rendering) and the API level (dependency injection).

User Roles

- **Admin**: Full system access, including user management and system settings.
- **Doctor**: Primary clinical access for patient diagnosis and treatment.
- **Nurse**: Clinical support and facility logistics management.

- **Patient**: Restricted access to personal health records and appointment booking.

- **Reception**: Administrative management of patients, scheduling, and billing.

- **Pharmacist**: Specialized access to medication inventory and dispensing.

- **Lab Technician**: Specialized access to diagnostic test management.

Module Permissions Matrix

Module	Admin	Doctor	Nurse	Patient	Reception
Patient Directory	CRUD	CR	CR	R (Self)	CR
Scheduling	CRUD	CR	-	CR (Self)	CR
Medical Records	CRUD	CR	R	R (Self)	-
Billing & Invoices	CRUD	-	-	R (Self)	CR
Pharmacy	CRUD	R	CR	-	R
Lab & Diagnostics	CRUD	CR	CR	-	-
Staff Management	CRUD	-	-	-	-
Inventory	CRUD	R	CR	-	CR
Blood Bank	CRUD	CR	CR	-	CR

Bed Management	CRUD	R	CR	-	CR

Legend:

- **C (Create)**: Ability to add new records.
- **R (Read)**: Ability to view existing records.
- **U (Update)**: Ability to modify existing records (restricted to **Admin** for most clinical data).
- **D (Delete)**: Ability to remove records (restricted to **Admin** only).

Data Isolation Policies

- **Patient Isolation**: Patients can *only* see records where their name matches the record's patient field.
- **Owner Isolation**: Non-admin staff may be restricted to viewing records they created, depending on hospital policy configurations in `auth_context.py`.

---

11. Environment Variables

Variables are defined in `.env`. Key configurations include:

- `ENV`: `development` or `production`.
- `HMS_SECRET_KEY`: High-entropy string for JWT signing.
- `DATABASE_URL`: Connection string (SQLite/Postgres).
- `ALLOWED_ORIGINS`: Domains allowed to access the API.
- `MAIL_SERVER/USER/PASS`: SMTP credentials for automated alerts.

---

11. Installation & Setup Guide

Prerequisites

- Node.js (v18+)
- Python (3.10+)
- PostgreSQL (Optional)

Step-by-Step Setup

1. **Clone the Repository**:
```bash
git clone
cd HMS
```
2. **Backend Setup**:
```bash
cd backend
python -m venv .venv
source .venv/bin/activate # or .venv\Scripts\activate on Windows
pip install -r requirements.txt
cp .env.example .env # Configure your variables
```
3. **Frontend Setup**:
```bash
cd ..
npm install
```
4. **Run Development Servers**:

- Backend: `uvicorn app.main:app --reload --port 8000`

- Frontend: `npm run dev`

---

12. Build & Deployment

Production Build

1. Build the React app: `npm run build`. This generates the `/dist` folder.
2. The FastAPI backend is configured to serve the `/dist` folder as static files in production.

Docker Deployment

The included `docker-compose.yml` sets up:

- **web**: The FastAPI + React app.
- **db**: PostgreSQL database.
- **Auto-migrations**: Handles schema updates via Alembic.

---

13. Workflow Explanation

Example: Patient Admission
1. **Receptionist** logs in and opens the **Patients** page.
2. Clicks "Add Patient", filling the form.
3. Frontend calls `POST /api/patients`.
4. Backend validates data via Pydantic, saves to DB via SQLAlchemy.
5. Backend triggers a **Webhook** to a nursing station dashboard.
6. **Nurse** sees the new patient in real-time on their dashboard via

WebSocket update.

---

14. Third-Party Integrations

- **n8n**: Workflow automation for appointment reminders and staff notifications.
- **SMTP (FastMail/Gmail)**: Sends invoices and password reset tokens.
- **Telegram (Optional)**: Can be integrated for doctor alerts via the `telegram_chat_id` field.

---

15. Challenges & Solutions

- **Challenge**: Performance lag with many modules.
- **Solution**: Implemented React `Suspense` and `lazy` loading for all route-level components.
- **Challenge**: Data security across roles.

- **Solution**: Unified Role-Based Access Control (RBAC) at both the Frontend (UI visibility) and Backend (API endpoint security).

- **Challenge**: Consistent UI across complex data tables.

- **Solution**: Developed the "Bones" metadata system to ensure all tables share the same filtering, pagination, and styling logic.

---

16. Future Improvements

- **Mobile Application**: Flutter or React Native integration using the existing REST API.

- **AI Integration**: Diagnostic assistance based on EMR data.

- **DICOM Viewer**: Support for viewing X-rays and MRI scans directly in the browser.

- **Multi-Hospital Support**: Refactoring schema for true multi-tenant "SaaS" architecture.

---

17. Conclusion

The Hospital Management System is a robust, scalable, and modern solution for healthcare facilities. Its modular architecture, combined with a secure backend and a metadata-driven frontend, makes it easy to maintain and extend for future clinical needs.